SPRINT 3 · BRIEF 2 · V1

AtlasRH : gestion des ressources humaines

Développement de l'API REST avec Spring Boot

Durée	2 semaines (du 19/10/2026 au 30/10/2026)
Mode de travail	Individuel
Livrable 1	Vendredi 23/10/2026 : la partie Responsable RH et la conception du projet
Livrable 2	Vendredi 30/10/2026 : le projet complet et la présentation

Contexte

AtlasRH est une entreprise de 250 collaborateurs. Aujourd'hui, ses démarches RH passent par e-mails et tableurs : fiches d'employés dispersées, contrats perdus, demandes de congé oubliées et soldes faux.

En tant que développeur Java, vous êtes chargé de développer la **première version (V1) de l'API REST du système RH d'AtlasRH**. Trois profils l'utilisent : le **responsable RH** gère les départements, les employés et leurs contrats, le **manager** valide les demandes de congé de son équipe, et chaque **employé** fait ses demandes et consulte son solde.

Une architecture propre en couches est indispensable.

Module Responsable RH

US1 : Gérer les départements

- Créer, modifier, consulter et supprimer un département (nom, description).
- Le nom d'un département est unique.
- Un département qui contient des employés ne peut pas être supprimé.

US2 : Gérer les employés

- Créer la fiche d'un employé : matricule, nom, prénom, e-mail, poste, date d'embauche, département et manager (un autre employé).
- Modifier et consulter une fiche.
- Lister les employés avec pagination, tri et recherche par nom ou par département.
- Consulter l'équipe d'un manager.

US3 : Gérer les contrats

- Créer un contrat pour un employé : type (CDI, CDD, STAGE), date de début, date de fin et salaire brut.
- Terminer un contrat en cours.
- Consulter l'historique des contrats d'un employé.

Module Manager

US4 : Valider les demandes de congé de mon équipe

- Voir les demandes en attente des employés de mon équipe.
- Approuver ou refuser une demande, avec un commentaire.

Module Employé

US5 : Faire une demande de congé

- Créer une demande : type de congé, date de début, date de fin et motif.
- Consulter mes demandes, avec filtre par statut (EN_ATTENTE, APPROUVEE, REFUSEE, ANNULEE).
- Annuler une demande tant qu'elle est en attente.

US6 : Consulter mon solde de congés

- Voir mon solde restant pour chaque type de congé.
- Voir mon ancienneté, calculée en années et en mois à partir de ma date d'embauche.

Modèle métier

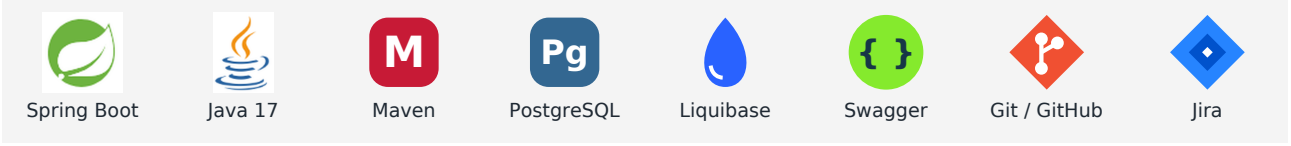
Cinq entités : `Department`, `Employee`, `Contract`, `LeaveType`, `LeaveRequest`. Deux types de congé existent dès le départ : **annuel** et **exceptionnel**, chacun avec un nombre de jours par an.

Règles de gestion

1. Le matricule et l'e-mail d'un employé sont uniques.
2. Un employé ne peut pas être son propre manager.
3. Un employé n'a qu'un seul contrat en cours. Un CDD ou un STAGE doit avoir une date de fin.
4. Le nombre de jours d'une demande est calculé en **jours ouvrés** (lundi à vendredi, dates comprises). La date de fin ne peut pas être avant la date de début, et deux demandes d'un même employé ne peuvent pas se chevaucher.
5. **Le solde n'est jamais saisi à la main** : solde restant = jours par an du type – jours des demandes approuvées de l'année. Une demande qui dépasse le solde restant est refusée.
6. Une décision n'est possible que sur une demande EN_ATTENTE, et un refus exige un commentaire. Seul l'employé peut annuler sa demande.

Exigences techniques

- Maven, Java 17, Spring Boot, API REST au format JSON.
- Spring Data JPA avec PostgreSQL et cinq entités : `Department`, `Employee`, `Contract`, `LeaveType`, `LeaveRequest`.
- Schéma de base de données géré par Liquibase, avec des données de démonstration fictives.
- Architecture en couches : controller, service, repository, entités. Les entités ne sortent jamais de la couche service : l'API échange des DTO, convertis avec MapStruct.
- Validation des données avec Bean Validation, gestion centralisée des exceptions et codes HTTP adaptés (200, 201, 400, 404, 409).
- Pagination et tri (`Pageable`) sur les listes importantes.
- Documentation de l'API avec Swagger (OpenAPI) : chaque endpoint est décrit.
- Logs avec SLF4J, configuration en YAML.
- Pas d'authentification pour l'instant : les rôles servent seulement à organiser les fonctionnalités.



Bonus

- Envoyer un e-mail (SMTP) à l'employé quand sa demande est approuvée ou refusée.
- Afficher l'organigramme de l'entreprise sous forme d'arbre hiérarchique.
- Enregistrer automatiquement les dates de création et de modification (JPA Auditing).

Déroulement : deux livrables

Le projet se construit en deux étapes, une par semaine. Vous commencez par la partie Responsable RH, puis vous ajoutez le Manager et l'Employé.

Livrable 1 : la partie Responsable RH (vendredi 23/10/2026)

- **Partie Responsable RH fonctionnelle** (US1 à US3), testée avec Swagger.
- **Diagramme de classes** de l'ensemble du projet : les cinq entités, leurs attributs, les relations et les cardinalités.
- **Planification Jira** : epics, user stories avec critères d'acceptation et tâches.
- **README initial** : comment lancer le projet, et l'architecture en couches expliquée (packages et conventions).

Livrable 2 : le projet complet (vendredi 30/10/2026)

- **Parties Manager et Employé** (US4 à US6), avec leurs règles de gestion.
- **Jira et README mis à jour** : chaque fonctionnalité est expliquée (à quoi elle sert, règles de gestion, exemple de requête et de réponse).
- **Diagramme de classes final**.
- **Présentation avec démonstration** de 15 minutes, suivie de questions.

Travail individuel

- Le code doit être propre et bien structuré : noms clairs, méthodes courtes, aucune logique métier dans les controllers.
- Une branche Git par fonctionnalité, fusionnée dans la branche principale.
- Le board Jira est mis à jour chaque jour.
- Vous devez savoir expliquer l'ensemble du projet.